

Technical Architecture – Siffrum Platform

1. Technology Stack

The proposed system will be built using a modern full-stack architecture that supports scalability, security, and AI integration.

Frontend Layer

The frontend will be responsible for user interaction and data visualization dashboards.

- HTML5
- CSS3
- JavaScript
- React.js / Next.js (for dynamic UI)

Key functions:

- Loan dashboard visualization
- User data input forms
- Financial analytics graphs
- Repayment strategy display

2. Programming Language

Frontend: JavaScript

Backend: Python

Python is selected due to strong machine learning ecosystem, fast API development, and robust fintech/data analysis libraries.

- FastAPI (high performance API development)
- Django REST Framework

3. Backend Workflow

- User logs into the platform
- User enters loan and financial information
- Frontend sends request to backend APIs
- Backend processes and validates the data
- Data is stored in the database
- Machine learning models analyze repayment risk
- System generates recommendations and stress scores
- Results are returned to the user dashboard

4. API Structure

- POST /api/register
- POST /api/login
- GET /api/user-profile
- POST /api/add-loan
- GET /api/get-loans
- PUT /api/update-loan
- DELETE /api/delete-loan
- GET /api/stress-score
- GET /api/loan-summary
- GET /api/repayment-plan
- POST /api/simulate-snowball
- POST /api/simulate-avalanche
- POST /api/simulate-refinance

5. Database Design

Recommended Database: PostgreSQL
Behavioral logs and analytics: MongoDB

- Users Table: user_id, name, email, password_hash, monthly_income, employment_type, created_at
- Loans Table: loan_id, user_id, principal_amount, interest_rate, tenure_months, emi_amount, start_date, loan_status
- Transactions Table: transaction_id, user_id, loan_id, transaction_amount, category, date
- Stress Analysis Table: analysis_id, user_id, stress_score, default_probability, recommended_strategy, created_at

6. System Architecture Overview

- User Interface Layer
- API Gateway Layer
- Application Logic Layer
- Machine Learning Engine
- Data Storage Layer

System Architecture Diagram

Technical Architecture – Siffrum Platform

1. Technology Stack

The proposed system will be built using a modern full-stack architecture that supports scalability, security, and AI integration.

Frontend Layer

HTML5

CSS3

JavaScript

React.js / Next.js (for dynamic UI). TensorFlow for user data input forms, Financial analysis

Programming Language

The platform will primarily utilize the following programming languages:

Frontend

JavaScript

Backend

Python

2. Backend Workflow

The backend architecture will follow a client-server model where the frontend communicates with backend services through secure APIs.

System workflow:

1. User logs into the platform.
2. User enters loan and financial information.
3. Frontend sends request to backend APIs.
4. Backend processes and validates the data.
5. Data is stored in the database.
6. Machine learning models analyse repayment risk.
7. System generates recommendations and stress scores.
8. Results are returned to the user dashboard.

4. API Structure

The platform will use a RESTful API architecture for communication between frontend and backend.

Authentication APIs

POST /api/register

POST /api/login / financial information.

GET /api/user-profile

POST /api/apr-loan / erdetenainic-Lone

DELETE /api/dete-loan / created.at

5. System Architecture Overview

The platform will use a relational database system to store structured financial data.

